

Overcut.ai - Complete Project Documentation

Track: Generative AI / Multimodal AI

Domain: Motorsport Analytics / Enterprise AI

Duration: 12 weeks

Team: 1 Student (Self-Project / BYOP)

Level: Advanced

Stack: Python, React, FastAPI, FAISS, Hugging Face (CLIP / LLaVA), open_clip, Tailwind CSS, FastF1, Polars, DuckDB, uPlot, XGrammar

Constraint: Strict 8GB VRAM (RTX 5060) & 16GB RAM Local Execution Limits

1. Project Overview

Project Identity

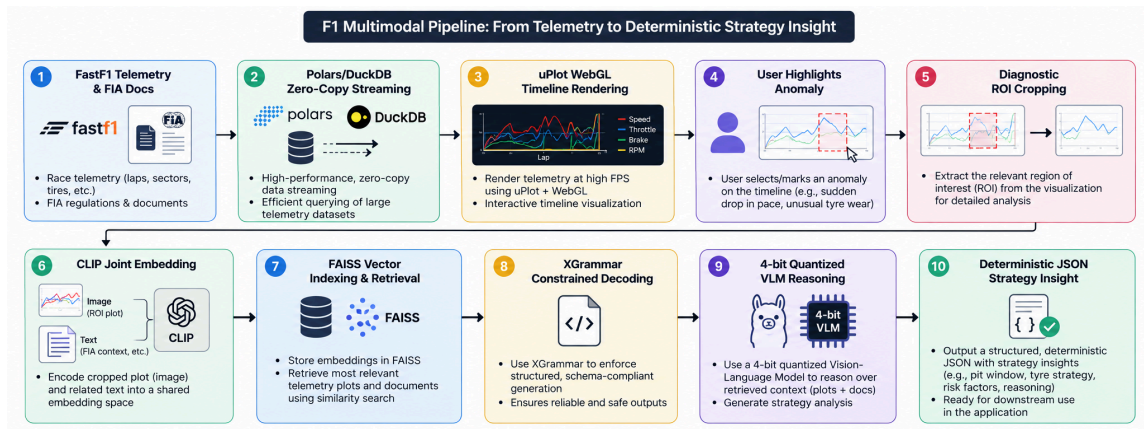
- **Track:** Generative AI
- **Domain:** Motorsport Analytics
- **Duration:** 12 weeks
- **Primary users:** Race Strategists, Performance Engineers, Motorsport Data Analysts
- **Constraint:** Must run locally on an 8GB VRAM GPU utilizing 4-bit model quantization.

Executive Summary

Overcut.ai is an end-to-end Generative Multimodal Observability Workspace designed for F1 race strategy and telemetry analysis. It ingests complex visual data such as FastF1 telemetry plots, tire

degradation charts, and FIA timing sheets alongside textual race reports. Replacing the initial conversational chatbot concept, the system now features a WebGL-accelerated interactive diagnostic timeline. When an engineer highlights a telemetry anomaly, the system utilizes Diagnostic Region of Interest (ROI) Cropping and XGrammar constrained decoding to output deterministic JSON strategy insights. By utilizing joint vision-language embeddings and a quantized Vision-Language Model (VLM), the system provides human engineers with factually grounded, visual-based diagnostics. The platform is intentionally designed as a decision-support system rather than an autonomous race engineer. A human strategist remains responsible for all final pit-wall decisions.

Core Product Flow



MVP (Minimum Viable Product)

1. Automated FastF1 data ingestion utilizing Polars/DuckDB for memory-efficient streaming.
2. WebGL (uPlot) frontend timeline for high-frequency data rendering without DOM lag.
3. Diagnostic ROI Cropping engine to slice and isolate specific temporal anomalies.
4. Joint text and image embedding pipeline using CLIP.
5. FAISS-based vector search index creation.
6. Local 4-bit VLM integration (e.g., LLaVA-1.5-7B) for multimodal reasoning.

7. XGrammar integration for strict JSON schema enforcement and deterministic output.
8. Display of retrieved visual evidence alongside AI-generated diagnostic cards.
9. Basic audit trail of system prompts and retrieved contexts.

Out of Scope for MVP

- Real-time live telemetry ingestion during an active Grand Prix (relies on historical FastF1 session data).
- Autonomous strategy execution or simulation software integration.
- Training a foundational Vision-Language Model from scratch.

2. Business Requirements Document (BRD)

2.1 Problem Statement

Modern F1 strategy teams and performance engineers spend significant time cross-referencing visual telemetry plots (e.g., speed traces, brake deltas) with unstructured text data (team radio transcripts, FIA directives). Standard text-based AI (OCR + LLM) destroys the spatial relationships embedded in visual charts, rendering current enterprise RAG solutions useless for telemetry analytics. Furthermore, attempting to solve this by feeding standard Vision-Language Models full, uncropped dashboards introduces "visual token dominance," which causes memory exhaustion on 8GB edge GPUs and induces severe AI hallucination.

2.2 Vision

Build an AI-assisted observability workspace that preserves the structural integrity of visual telemetry, allowing engineers to highlight specific historical race anomalies on a high-performance timeline to receive instant, structured diagnostic insights backed directly by visual proof.

2.3 Objectives

- Eliminate the "OCR bottleneck" by analyzing raw visual plots directly via Vision-Language Models.
- Implement Diagnostic Region of Interest (ROI) Cropping to isolate specific temporal anomalies, preventing token exhaustion and visual hallucinations.
- Generate deterministic, schema-validated JSON diagnostics instead of probabilistic prose to directly power the frontend UI.
- Maintain peak performance strictly within localized hardware constraints (8GB VRAM and 16GB system RAM).

2.4 Personas

Persona	Goals	Pain Points
Race Strategist	Identify undercut/overcut windows quickly.	Too many fragmented data screens during a stint, cannot quickly isolate anomalies.
Performance Engineer	Compare driver braking/throttle traces visually.	Extracting actionable, structured insights from raw visual curves takes too much time..
Data Analyst	Build rapid post-race debrief reports.	Standard telemetry systems crash browser DOMs, manually correlating lap times is tedious.

2.5 Business Requirements

ID	Requirement	Priority	Acceptance Criteria
BR-001	The system ingests visual plots and text reports.	Must	Files are processed and chunked successfully utilizing Polars/DuckDB zero-copy streaming.
BR-002	The system maps text and cropped images to a shared vector space.	Must	CLIP successfully embeds both modalities into FAISS.
BR-003	The system retrieves relevant visual context for queries.	Must	Highlighting a timeline anomaly correctly returns the historical visual trace and technical context.
BR-004	The system generates deterministic, explainable strategy insights.	Must	VLM outputs strict, schema-validated JSON referencing the micro-cropped visual chart.

ID	Requirement	Priority	Acceptance Criteria
BR-005	The system fits hardware constraints.	Must	Peak VRAM usage remains strictly under 7.5GB during inference.

2.6 Non-Functional Requirements

- **Hardware Constraints (GPU):** Must utilize INT4 quantization via 'bitsandbytes' to ensure the LLaVA-1.5-7B model does not trigger Out-Of-Memory (OOM) errors and stays strictly under the 8GB VRAM limit of the RTX 5060.
- **Hardware Constraints (RAM):** The data ingestion pipeline must utilize out-of-core memory management (Polars/DuckDB) to prevent thrashing the 16GB system RAM limit during long-horizon session loads.
- **Accuracy (Anti-Hallucination):** The VLM must default to a null JSON state or "Insufficient Data" if the required visual plot is not retrieved from the FAISS context window.
- **Determinism:** The backend must utilize XGrammar to precompute context-free grammar pushdown automata, guaranteeing the VLM outputs strict JSON without programmatic retry loops.
- **Auditability:** Every generated diagnostic UI card must visually display the exact micro-cropped image used by the VLM to formulate the response.

2.7 Success Metrics

- **Retrieval Recall@K:** The correct historical telemetry plot and context appears in the top 3 retrieved FAISS results >85% of the time.
- **Schema Adherence Rate:** 100% of VLM inference responses successfully parse as valid JSON on the first pass.

- **Inference Latency:** Time-To-First-Token (TTFT) is under 5 seconds on local hardware, accelerated by the XGrammar token mask cache.
- **Faithfulness Score:** High human-evaluation agreement that the structured JSON output correctly interprets the specific coordinate logic of the micro-cropped chart without hallucinating unprovided data.
- **Resource Ceilings:** The complete end-to-end pipeline (Data Ingestion + VLM Inference) never exceeds 7.5GB of VRAM and 14GB of system RAM concurrently.

3. Product Requirements Document (PRD)

3.1 Product Goal

Create an enterprise-grade multimodal F1 observability workspace that converts high-frequency visual telemetry, timing sheets, and historical unstructured data into deterministic, structured diagnostic UI cards using Region of Interest (ROI) cropping and constrained VLM decoding.

3.2 Feature Priorities

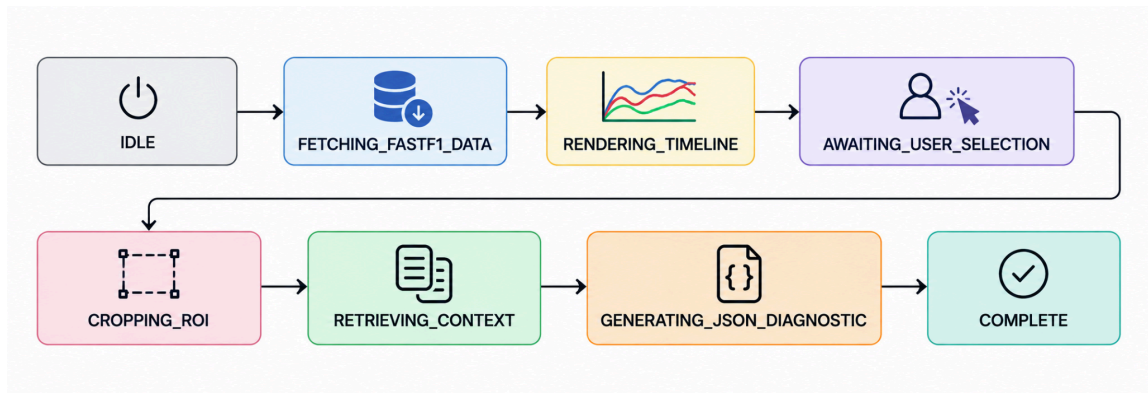
Feature	Priority
FastF1 Session Data Ingestion via Polars/DuckDB	Must
WebGL (uPlot) Interactive Timeline Rendering	Must
Diagnostic ROI Cropping Engine	Must

Feature	Priority
Text/Image Chunking & CLIP Joint Embedding	Must
FAISS Vector Indexing & Similarity Search	Must
4-bit VLM Generative Inference (LLaVA)	Must
XGrammar Constrained JSON Decoding	Must
Visual Evidence/Citation Display	Must

3.3 User Stories

- **US-001:** As a race strategist, I want to drag a selection box over a telemetry anomaly on an interactive WebGL timeline so that the system isolates the specific temporal event for targeted AI analysis.
- **US-002:** As a data engineer, I want to ingest FastF1 session data via an automated Polars/DuckDB pipeline so that the system can process and embed new telemetry plots without thrashing the 16GB system RAM limit.
- **US-003:** As an analyst, I want the AI's generated response to output as structured JSON rendering into diagnostic UI cards, so that I can independently verify the VLM's claims against the micro-cropped plot.
- **US-004:** As an auditor, I want a history of all system prompts, generated JSON schemas, and retrieved contexts so that I can debug AI hallucinations or retrieval failures.

3.4 Product States



3.5 Product Principles

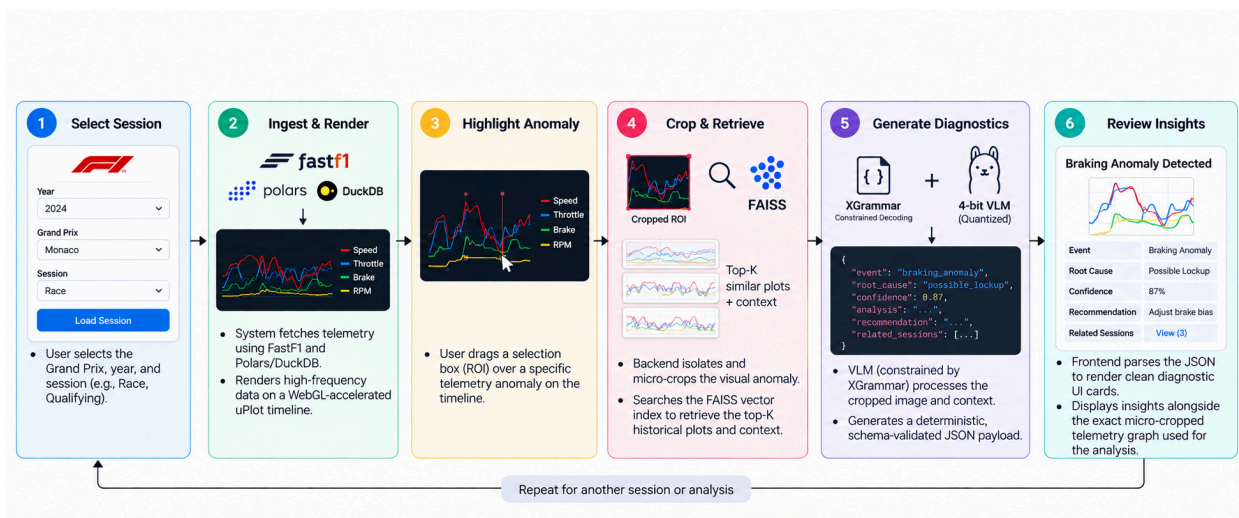
- **Deterministic Output over Probabilistic Prose:** The VLM must generate strict, schema-validated JSON utilizing XGrammar to directly power the frontend diagnostic UI, preventing unpredictable conversational text.
- **Targeted Perception:** The system must never feed a full-screen dashboard to the VLM; it must always crop to the specific Region of Interest to prevent visual token dominance and hallucination.
- **Evidence Before Explanation:** The system must retrieve the historical visual plot and context from FAISS before the VLM is allowed to reason about the anomaly.
- **Clear Uncertainty:** If FAISS fails to retrieve relevant telemetry, the VLM must output a null JSON state indicating "Insufficient data" rather than fabricating lap times or physics.
- **Visual Traceability:** Every GenAI output must visually display the exact micro-cropped data chart used to generate the insight.

4. UX Requirements

4.1 Information Architecture

1. Session Selection & Ingestion Hub
2. Enterprise Observability Workspace (WebGL/uPlot Timeline Main UI)
3. Diagnostic ROI Tool (Timeline Selection Box)
4. Visual Evidence & Diagnostic Card Viewer (Split-screen)
5. FAISS Index Explorer (Debug View)

4.2 Primary User Flow



4.3 Key Screen Requirements

- **Session Ingestion**: Progress bars for FastF1 API downloads, zero-copy Polars processing, timeline rendering, and CLIP embedding generation.
- **Enterprise Observability Timeline**: A high-performance WebGL (uPlot) canvas that handles 100Hz telemetry streams without browser DOM lag, featuring a drag-and-drop Region of Interest (ROI) bounding box for anomaly selection.
- **Diagnostic Evidence Viewer**: A dynamic side-panel that prominently displays the structured JSON diagnostic UI cards

(e.g., risk factors, root causes) directly above the specific micro-cropped telemetry graph cited by the AI's current response.

5. Technical Requirement Document (TRD)

5.1 Proposed Architecture

- **Frontend:** React, Tailwind CSS, Axios, uPlot (WebGL).
- **Backend/API:** Python, FastAPI, XGrammar.
- **Data Pipeline:** FastF1, Polars, DuckDB, Matplotlib (for Region of Interest generation).
- **Embeddings & Vector DB:** Hugging Face open_clip, FAISS.
- **Generative AI:** Quantized Vision-Language Model (e.g., LLaVA-1.5-7B) via Hugging Face Transformers & bitsandbytes (4-bit INT4).

5.2 Technology Rationale

- **FastAPI:** Highly performant, asynchronous, and the industry standard for serving Python-based ML/AI endpoints.
- **FastF1:** The definitive open-source library for extracting highly accurate F1A timing and telemetry data.
- **Polars / DuckDB:** Replaces Pandas to provide zero-copy, out-of-core memory management, ensuring the 16GB system RAM is not overwhelmed during high-density session telemetry loads.
- **uPlot / WebGL:** Standard frontend charting libraries fail and crash the browser DOM under 100Hz telemetry streams. uPlot

guarantees sub-pixel fidelity and smooth rendering for the interactive timeline.

- **XGrammar:** Precomputes context-free grammar pushdown automata to force the LLaVA model to output strictly validated JSON, bypassing the latency and failure rates of programmatic retry loops.
- **FAISS:** Extremely efficient for local, memory-constrained vector similarity search.
- **4-bit Quantization (bitsandbytes):** Strictly necessary to fit a 7B parameter multimodal model into the 8GB VRAM constraint of the RTX 5060.

5.3 Technical Requirements

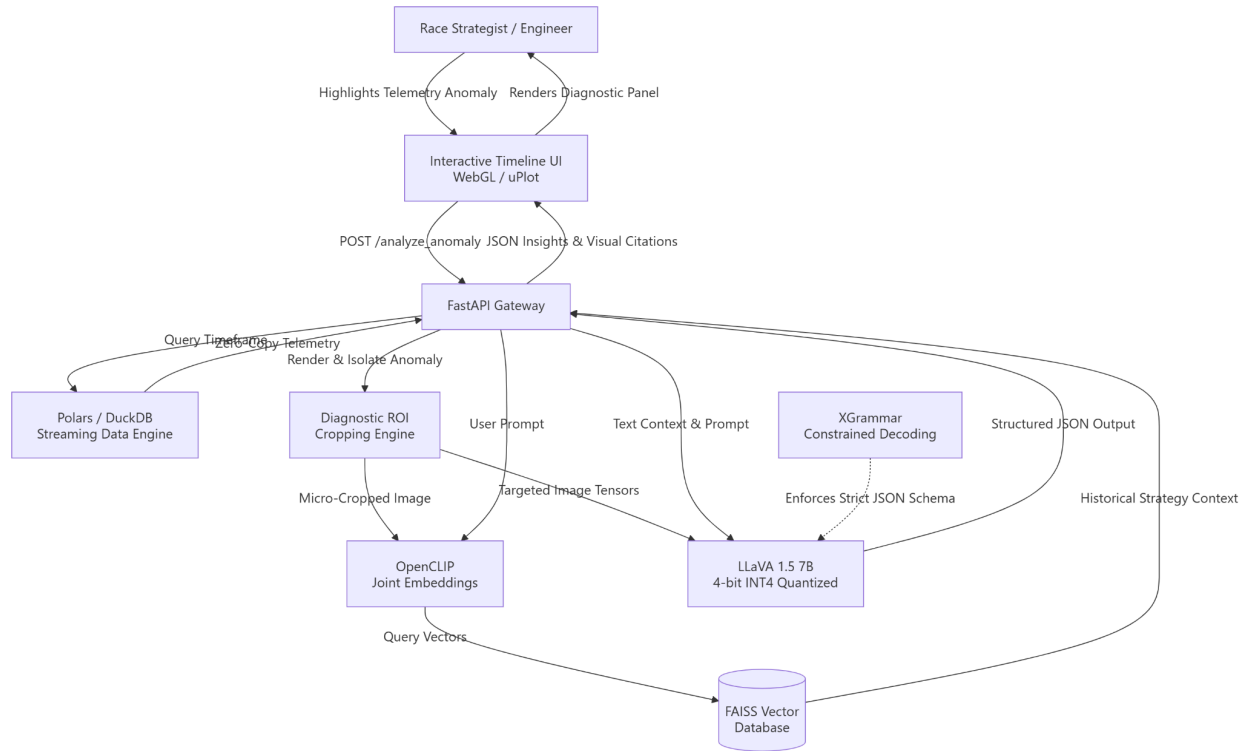
ID	Requirement	Priority	Maps to
TR-001	Async FastF1 data ingestion via Polars/DuckDB zero-copy streaming.	Must	US-002
TR-002	WebGL interactive timeline rendering and Diagnostic ROI Cropping pipeline.	Must	US-001
TR-003	CLIP multimodal embedding and FAISS Top-K retrieval API.	Must	US-001, US-004
TR-004	4-bit VLM inference endpoint featuring XGrammar constrained JSON decoding.	Must	US-003
TR-005	Frontend dynamic binding of JSON diagnostic cards alongside micro-cropped visual evidence.	Must	US-003

5.4 NFR (Non-Functional Requirements) Targets

- **VRAM Limit:** Maximum GPU memory allocation must strictly remain under 7.5GB during the VLM generation phase, enforced by feeding only micro-cropped ROI images rather than full dashboards.
- **System RAM Limit:** The complete data pipeline and API execution must not exceed 14GB of memory usage to protect the 16GB system RAM ceiling.
- **Retrieval Speed:** FAISS vector search and context assembly must complete in < 1.5 seconds.
- **Idempotent Processing:** FastF1 local cache must be utilized to prevent redownloading the same session data and hitting FIA API rate limits.

6. High Level Design (HLD)

6.1 Architecture



6.2 Components

Component	Responsibility
React / WebGL UI	Renders the high-frequency uPlot interactive timeline, handles ROI bounding-box selection, and parses JSON to display diagnostic UI cards.
FastAPI	Request routing, execution orchestration, and hardware memory management.
Polars / FastF1 Engine	Fetching FIA session data using zero-copy streaming engines to prevent system RAM bottlenecks, and

Component	Responsibility
	executing Diagnostic ROI Cropping to isolate anomalies.
CLIP / FAISS	Mapping micro-cropped visual anomalies and text context into a shared vector space, and executing rapid similarity searches.
VLM (LLaVA) & XGrammar	Synthesizing insights from the micro-cropped visual ROI and forcing the 4-bit quantized model to output deterministic, schema-validated JSON.

6.3 End-to-end Data Flow

1. User selects a Grand Prix session; the system fetches telemetry data via FastF1 and Polars/DuckDB zero-copy streaming.
2. System renders the high-frequency telemetry data onto a WebGL-accelerated (uPlot) interactive timeline.
3. User drags a selection box (Region of Interest) over a specific telemetry anomaly on the timeline rather than typing a text query.
4. The backend isolates and micro-crops the precise visual anomaly (Diagnostic ROI Cropping).
5. The system embeds the micro-cropped image using CLIP and searches the FAISS index.
6. FAISS returns the Top-K relevant historical plots and textual strategy context.
7. The micro-cropped image and retrieved context are passed to the 4-bit quantized VLM (LLaVA).
8. XGrammar intercepts the generation, constraining the VLM to output a deterministic, schema-validated JSON payload.
9. The backend streams the structured JSON back to the React UI, which renders diagnostic UI cards alongside the exact micro-cropped source image.

7. Database / Data Design

Since this is a Retrieval-Augmented Generation system, the data architecture is split between a standard document database for metadata/history and a vector database for similarity search.

7.1 Core Collections (MongoDB or SQLite)

`sessions`

Tracks ingested FastF1 Grand Prix sessions.

- `_id / session_id`: Unique identifier (e.g., 2023_Monaco_Race)
- `year`: Championship year
- `event_name`: Grand Prix name
- `session_type`: Race, Qualifying, Sprint
- `ingestion_status`: PENDING, COMPLETED, FAILED
- `created_at`: Timestamp

`visual_artifacts`

Maps historical generated FastF1 plots to their FAISS vector IDs for retrieval context.

- `_id / artifact_id`: Unique ID for the generated plot
- `session_id`: Reference to the parent session
- `artifact_type`: speed_trace, tire_degradation, track_map, lap_times
- `storage_path`: Local file path to the generated PNG/JPG
- `faiss_index_id`: The corresponding integer ID in the FAISS index
- `metadata`: Driver numbers, lap ranges, corner numbers

`anomaly_diagnostics`

Audit log of timeline anomalies and deterministic VLM JSON outputs.

- `_id`: Unique diagnostic ID
- `session_id`: Active session context
- `roi_bounds`: The temporal/spatial coordinates selected on the interactive timeline (e.g., `{"start_lap": 14, "end_lap": 16, "telemetry_channels": ["speed", "throttle"]}`)
- `retrieved_artifacts`: Array of `artifact_ids` retrieved by FAISS for context
- `diagnostic_json`: The structured JSON payload generated by the VLM (e.g., `root_cause`, `confidence_score`, `sensor_delta`)
- `inference_time_ms`: Tracking TTFT and total generation latency
- `timestamp`: Execution time

7.2 Vector Index (FAISS)

- Storage: Local `.index` file to ensure zero cloud storage costs and maintain strict local execution.
- Dimensions: 512 or 768 dimensions (depending on the specific `open_clip` model chosen).
- Mapping: Vector IDs map directly to `faiss_index_id` in the `visual_artifacts` collection.

7.3 Data Lifecycle

User requests F1 Session → FastF1 fetches raw data via Polars/DuckDB → System renders WebGL timeline → User highlights ROI anomaly on the timeline → Backend micro-crops the visual anomaly → CLIP generates embeddings for the cropped image → Vectors search FAISS for historical context → XGrammar constrained VLM processes the cropped image and retrieved context → VLM outputs structured JSON diagnostic → Ephemeral VLM context cleared from 8GB VRAM to prevent fragmentation.

8. API Specification

Base URL : /api/v1

8.1 Ingestion Endpoints

POST /sessions/ingest

Triggers the FastF1 data pipeline to download session data and stream it through the Polars/DuckDB zero-copy engine for timeline readiness.

- Payload: {"year": 2023, "event": "Monza", "session": "Race"}
- Response: {"session_id": "2023_Monza_Race", "status": "processing"}

GET /sessions

Lists all successfully ingested and indexed F1 sessions available for timeline viewing and anomaly querying.

8.2 Query Endpoints

POST /analyze_anomaly

The primary multimodal observability endpoint. Triggers the Diagnostic ROI Crop, executes the FAISS vector search, and runs the XGrammar-constrained VLM generation.

- Payload:

JSON

```
{
  "session_id": "2023_Monza_Race",
  "roi_bounds": {
    "start_lap": 14,
    "end_lap": 16,
    "telemetry_channels": ["speed", "brake"]
  }
}
```

```
}
```

- Response:

JSON

```
{
  "diagnostic_id": "diag_98765",
  "diagnostic_json": {
    "summary": "Carlos Sainz suffered a rear-left lockup entering
Turn 1, evident from the sharp drop in the speed trace and
simultaneous brake pressure spike.",
    "root_cause": "brake_bias_imbalance",
    "risk_factors": ["Tire flat-spotting", "Sector 1 time
degradation"],
    "confidence_score": 0.92
  },
  "cropped_roi_url":
"/static/roi/2023_Monza_Race_lap14-16_crop.png",
  "evidence_artifacts": [
    {"artifact_id": "art_045", "url":
"/static/plots/historical_monza_lockup.png"}
  ],
  "inference_time_ms": 4250
}
```

8.3 Standard Errors

JSON

```
{
  "error": {
    "code": "ROI_CROPPING_FAILED",
```

```
    "message": "The requested timeline coordinates exceed the  
    bounds of the active session data.",  
    "requestId": "req_555"  
  }  
}
```

Follows standard HTTP status codes:

- 400 (Bad Request / Invalid ROI Bounds)
- 404 (Session Not Found)
- 422 (XGrammar Validation Failure)
- 500 (Server / CUDA Out-Of-Memory Error)

9. Low Level Design (LLD)

9.1 Backend Modules (FastAPI/Python)

None

```
app/  
├─ api/           # FastAPI routers (endpoints)  
├─ services/      # Core business logic (anomaly routing,  
generation)  
├─ data_engine/   # FastF1 fetching, Polars zero-copy streaming,  
and Matplotlib ROI cropping  
├─ ai/            # CLIP embedding, LLaVA VLM orchestration, and  
XGrammar compilation  
├─ vector_store/  # FAISS index management  
├─ models/        # Pydantic schemas (Input validation and  
XGrammar output FSMs)  
└─ core/          # Config, VRAM/RAM hardware checks, logging
```

9.2 Key Interfaces

FastF1Processor

- `fetch_and_stream_session(year, event, session_type)`: Ingests FIA data using Polars/DuckDB for out-of-core memory management to protect system RAM.
- `generate_roi_crop(session_id, start_time, end_time, channels)`: Slices the specific temporal anomaly selected on the timeline and renders a micro-cropped image with concatenated axes.

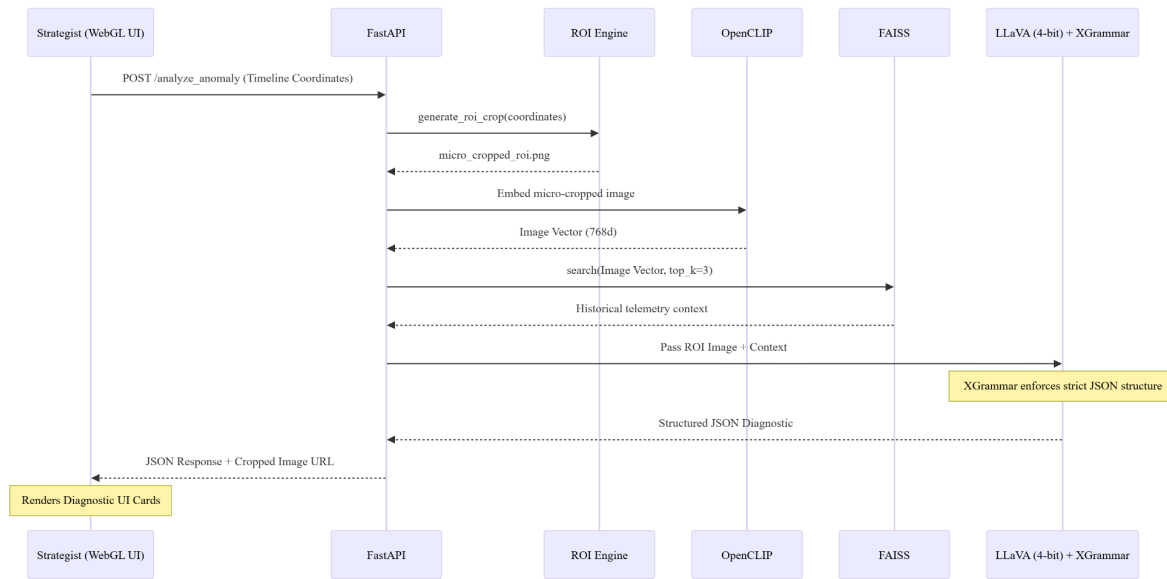
VectorEngine

- `embed_artifact(image_path, text_metadata)`: Uses `open_clip` to create joint embeddings.
- `search(roi_embedding, top_k=3)`: Searches FAISS using the vector of the micro-cropped anomaly to find historical structural precedents.

DiagnosticCopilot

- `build_context(retrieved_images, roi_crop)`: Formats the micro-cropped anomaly and historical visual context for the VLM.
- `generate_diagnostic_json(context)`: Executes 4-bit LLaVA inference constrained by XGrammar to guarantee a deterministic, JSON-formatted output.

9.3 Sequence Diagram (Anomaly Analysis Flow)



9.4 Error Handling and Hardware Validation

- VRAM Checks:** Before initializing LLaVA, the core module checks `torch.cuda.memory_allocated()`. If the requested inference risks exceeding the 7.5GB threshold of the RTX 5060, the system flushes the CUDA cache before generation.
- System RAM Checks:** The Polars ingestion pipeline utilizes `psutil` to monitor virtual memory, ensuring the zero-copy streaming does not exhaust the host machine's 16GB RAM limit.
- Output Schema Validation:** Instead of relying on programmatic retry loops, XGrammar pre-compiles the Pydantic schema into a finite state machine. This hardware-validates the VLM output at the token generation level, guaranteeing the AI cannot break the frontend UI with malformed markdown, trailing commas, or conversational hallucinations.

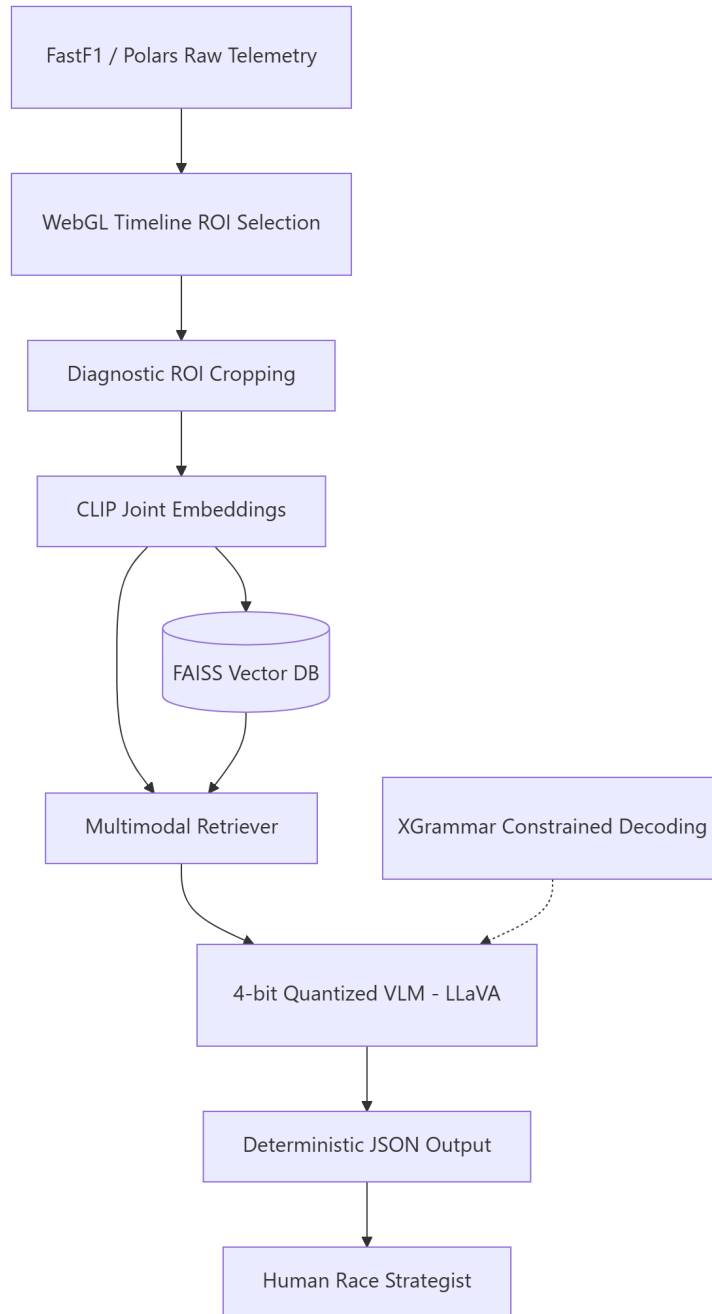
10. Generative AI Architecture

10.1 Why GenAI?

Traditional telemetry analysis requires human engineers to manually interpret overlapping time-series graphs. While standard text-based AI cannot comprehend spatial curves, generic Vision-Language Models (VLMs) also fail when fed full-screen dashboards, suffering from "visual token dominance" which causes severe memory exhaustion and hallucination.

To solve this, Overcut.ai uses a **Targeted Perception** architecture. Generative AI is utilized purely for fine-grained visual reasoning over micro-cropped anomalies. The system does not use the VLM to autonomously calculate race physics or make final pit-stop decisions; it acts as a deterministic decision-support engine.

10.2 Recommended AI Architecture



10.3 Multimodal RAG Pipeline

FastF1 Polars Ingestion → WebGL Timeline Rendering → User Selects Timeline Anomaly (ROI) → Diagnostic ROI Cropping isolates the visual slice → CLIP Image/Text Embeddings → FAISS Vector Store →

Similarity Search → Micro-Cropped Image + Context passed to Inference → XGrammar Constrained VLM Synthesis.

10.4 Prompt Structure

System Instructions:

- Act as an F1 enterprise observability diagnostic assistant.
- Use only the provided micro-cropped telemetry plot and historical session data.
- Never invent lap times, tire compounds, or driver actions not visible in the provided context.
- If the micro-cropped plot does not contain sufficient data, output a null JSON state.
- You must strictly adhere to the provided JSON schema.

User/Context input:

- Micro-cropped telemetry image tensor (ROI).
- Retrieved historical visual plots from FAISS.
- Accompanying axis metadata and context.

10.5 Structured Output

The VLM no longer outputs unstructured text. XGrammar forces the model to generate the following schema:

JSON

```
{  
  "summary": "...",  
  "root_cause": "...",  
  "risk_factors": [],  
  "missing_telemetry": [],
```

```
"evidence_references": [  
  "artifact_id_045"  
],  
"confidence_score": 0.0  
}
```

10.6 Guardrails

- **Deterministic Schema Validation:** Instead of programmatic retry loops, XGrammar precomputes context-free grammar pushdown automata to guarantee 100% syntactically valid JSON at the token generation level.
- **Targeted Perception (VRAM Protection):** Diagnostic ROI Cropping prevents the "high bandwidth memory (HBM) tax" by strictly limiting the visual tokens processed per query, preventing the RTX 5060's 8GB VRAM from overflowing.
- **Evidence Citation Checking:** The VLM is programmatically restricted from citing an artifact_id that was not successfully retrieved by FAISS.

10.7 AI Safety and Evaluation

Treat team radio transcripts and external text as untrusted input to prevent prompt injection. Evaluate the system on:

- **Schema Adherence Rate:** Does the VLM output parse as valid JSON on the very first pass without retry logic?
- **Groundedness:** Does the structured JSON accurately reflect the coordinate logic of the specific micro-cropped chart?
- **Hallucination Rate:** Did it invent a driver action or physics calculation not present in the localized visual?

- **Retrieval Accuracy:** Did FAISS successfully retrieve the correct historical structural precedent based on the cropped anomaly?

11. Security Design

11.1 Authentication and Authorization

Use token/session-based authentication mapped to motorsport roles:

- **STRATEGIST:** Read/Write access to all sessions, the interactive timeline, and anomaly diagnostic generation.
- **PERFORMANCE_ENGINEER:** Read/Write access to telemetry ingestion, Polars data engine rendering, and Diagnostic ROI Cropping.
- **ADMIN:** System architecture, FAISS vector index management, and document DB maintenance.

11.2 Data Protection and Privacy

- **Total Local Execution:** Because the VLM (LLaVA) runs entirely locally on the host machine's RTX 5060, proprietary team strategies, anomaly queries, and telemetry data never hit a cloud API, eliminating the risk of external data leakage.
- **Secret Management:** Keep system paths, local FastF1 cache directories, and any optional API keys in environment variables.

11.3 File Security and Input Validation

- **Directory Integrity:** Strictly validate file paths for the FastF1 cache and Polars/DuckDB streaming pipelines to prevent path/directory traversal attacks.
- **Storage Constraints:** Restrict accepted formats for local storage to .png or .jpg for the micro-cropped visual artifacts, and strictly .json for the XGrammar-enforced metadata.

- Prompt Injection Mitigation: By replacing conversational chat with a timeline ROI selection tool and forcing the VLM output through XGrammar's compiled pushdown automata, the system structurally isolates the AI from attempting to execute or return malicious prompt injections.

11.4 Important Product Rule

The system is an enterprise decision-support prototype engineered for observability and diagnosis. Final pit-stop and race strategy decisions must remain with an authorized human reviewer.

12. Testing Strategy

12.1 Test Levels

1. Unit Tests: FastF1 data extraction, Polars/DuckDB zero-copy data streaming, and Diagnostic ROI Cropping boundary logic.
2. Integration Tests: CLIP multimodal embedding pipeline, FAISS vector similarity search, and XGrammar pushdown automata compilation.
3. API Tests: FastAPI endpoints (/sessions/ingest, /analyze_anomaly).
4. End-to-End Tests: Full interactive user flow (WebGL uPlot timeline selection → ROI Crop → FAISS retrieval → VLM JSON generation → Diagnostic UI card render).
5. AI/ML Evaluation: VLM hallucination rate, schema adherence rate (via XGrammar), TTFT latency checks, and hardware memory profiling (VRAM and RAM ceilings).

12.2 Sample Test Cases

ID	Scenario	Expected Result	Priority
TEST-001	Valid FastF1 session load via Polars	Session cached, data streamed to WebGL timeline without system RAM thrashing.	High
TEST-002	Invalid F1 year/event	FastAPI returns a 404 validation error.	High
TEST-003	Hardware threshold check (VRAM/RAM)	Pipeline flushes cache or rejects load if GPU VRAM > 7.5GB or system RAM > 14GB.	High
TEST-004	Diagnostic ROI Crop boundary check	The cropping engine accurately isolates the user-selected temporal window and dynamically attaches Y-axis context.	High
TEST-005	FAISS visual similarity search hit	Returns relevant historical structural precedent artifact_ids.	High
TEST-006	XGrammar constrained decoding enforcement	VLM generates 100% syntactically valid JSON on the first pass; probability of generating markdown or prose is zero.	High
TEST-007	Out-of-bounds ROI timeline selection	API returns 400 Bad Requests with descriptive coordinate bounds error.	High
TEST-008	Engineer diagnostic audit logging	The complete JSON output and retrieved contexts are successfully saved to the anomaly_diagnostics database collection.	Medium

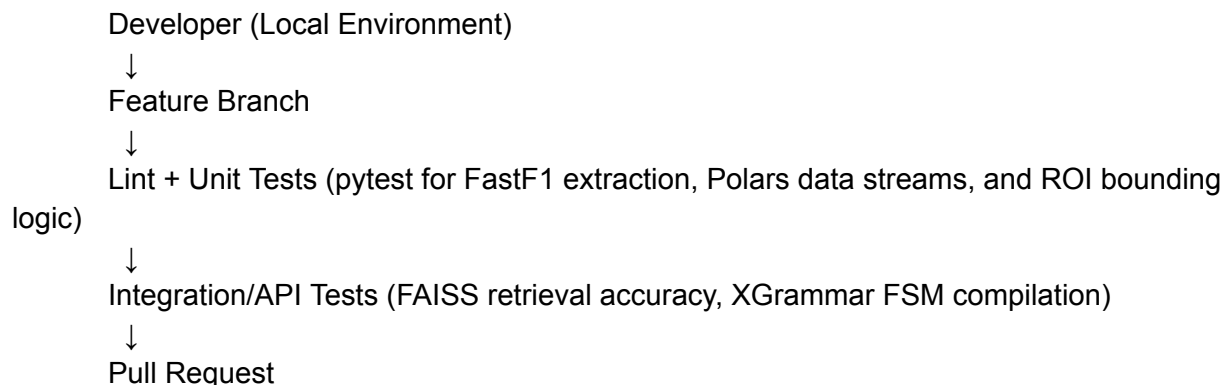
12.3 ML and GenAI Evaluation

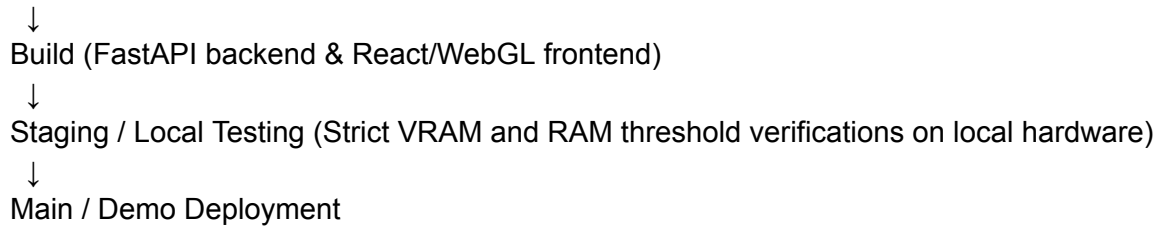
Create a labeled evaluation set of micro-cropped FastF1 anomalies (e.g., a known tire lock-up or undercut scenario). Evaluate the system on the following metrics:

- **Zero-Shot Schema Compliance:** Does XGrammar guarantee valid JSON output on the very first inference pass without requiring programmatic retry loops?
- **Groundedness:** Evaluate the factual consistency of the VLM against the provided micro-crop. Does the generated diagnostic match the specific coordinate logic of the visual trace?
- **Hallucination Rate:** Penalize the system for any unsupported statements, invented telemetry values, or fabricated driver actions that are not explicitly visible in the cropped ROI.
- **Hardware Profiling:** Log the maximum number of JSON tokens generated across various batch sizes to ensure the 4-bit LLaVA model never exceeds the 8GB RTX 5060 ceiling during peak inference.

13. CI/CD

13.1 Pipeline





13.2 Quality Gates

- Do not merge if FastF1 rendering scripts fail to produce accurate, micro-cropped ROI images.
- Do not merge if VRAM memory leak tests exceed the 7.5GB threshold on the RTX 5060, or if Polars ingestion triggers system RAM usage above 14GB.
- Do not merge if XGrammar schema adherence tests fail (VLM output must be 100% valid JSON without retry logic).
- Ensure no API keys, FastF1 cache paths, or local file system paths are hardcoded into the repository.

14. Observability

14.1 Logs

Log the following critical metrics:

- Request ID, Session ID, and Diagnostic ID.
- FastF1 and Polars extraction status (e.g., `fetching_telemetry`, `zero_copy_streaming_complete`).
- ROI Cropping coordinates and execution status.
- FAISS retrieval time.
- Hardware allocation spikes: GPU VRAM (via `torch.cuda.memory_allocated()`) and System RAM (via `psutil`).
- Error codes (e.g., `OUT_OF_MEMORY`, `ROI_CROPPING_FAILED`, `XGRAMMAR_COMPILE_ERROR`).

Strict Rule: Do not log raw base64 image strings, massive tensor arrays, or full telemetry DataFrames to prevent log bloat and disk exhaustion.

14.2 Metrics

- **API Latency:** Target total response times for end-to-end anomaly queries.
- **Time-To-First-Token (TTFT):** Essential for measuring the perceived speed of the quantized VLM alongside any XGrammar token-masking precomputation overhead.
- **Hardware Usage:** Peak GPU VRAM and System RAM utilized per query.
- **Schema Parse Success Rate:** Monitored to ensure XGrammar enforcement maintains a 100% syntactically valid JSON output rate.

14.3 Tracing

Trace path: WebGL Frontend ROI Selection → FastAPI routing → Polars Data Extraction → Diagnostic ROI Cropping → FAISS vector search → Image context assembly → XGrammar-constrained Quantized VLM inference → Structured JSON response payload → WebGL Diagnostic Card Render.

15. Deployment Architecture

15.1 Environments

Local Development → Local Testing (Strict 8GB VRAM / 16GB RAM verifications) → Demo Deployment

15.2 Suggested Deployment

Because of the strict 8GB VRAM limitation, this system is optimized for a powerful local environment rather than expensive cloud deployment:

- Frontend: React web application utilizing WebGL-accelerated uPlot for the interactive timeline, served locally via Vite/Node.
- Backend: FastAPI running in a local Python virtual environment to maintain direct access to the host machine's RTX 5060 GPU and system memory allocation via psutil.
- Data Engine: Polars/DuckDB handling out-of-core data streaming to protect system RAM.
- Vector Database: FAISS .index files persisted on the local disk.
- Generative AI: Hugging Face model weights cached locally and loaded into VRAM using bitsandbytes 4-bit quantization, with inference strictly governed by XGrammar precompiled schemas.

15.3 GPU Constraint Strategy

1. Quantization: Use LLaVA-1.5-7B quantized to 4-bit INT4 formats to fit within the 8GB VRAM ceiling.
2. Targeted Perception: Enforce Diagnostic ROI Cropping to minimize the visual token payload sent to the VLM, preventing high-bandwidth memory exhaustion.
3. Zero-Copy Memory Management: Utilize Polars and DuckDB instead of Pandas to prevent memory thrashing on the 16GB system RAM during long-horizon session ingestion.

4. Garbage Collection: Clear the CUDA cache (`torch.cuda.empty_cache()`) aggressively between timeline anomaly queries to prevent fragmentation.

16. Cost Analysis

16.1 Cost Model

This architecture is designed to be highly cost-optimized for local execution. By executing inference on the host machine and utilizing permissive open-source libraries, cloud compute costs are completely eliminated.

- Data Acquisition: \$0 (FastF1 is open-source and free, distributed under the MIT License).
- Data Engine: \$0 (Polars and DuckDB are open-source).
- Vector Search: \$0 (FAISS runs locally).
- LLM Inference: \$0 (LLaVA code is Apache-2.0 licensed, running entirely on local GPU execution).
- Storage: Minimal local SSD storage for FastF1 caching, FAISS indexes, and micro-cropped ROI .png files.

16.2 Optimization

- Cache FastF1 session downloads locally to prevent repetitive network requests and FIA API rate-limiting.
- Only pass the isolated, micro-cropped ROI image and the top 1 or 2 highest-scoring FAISS context images to the VLM. This minimizes the prompt context window, accelerating Time-To-First-Token (TTFT) and keeping the VRAM footprint under 7.5GB.

17. 12-Week Project Roadmap

Phase	Weeks	Deliverables
Research & Data	1-2	FastF1 library mastery, Polars zero-copy streaming setup, and ROI boundary cropping logic.
Architecture	3	Finalizing TRD, HLD, DB design, and hardware constraint verifications.
Backend Foundation	4-5	FastAPI setup, MongoDB/SQLite integration, and XGrammar schema definitions.
Embeddings & Search	6-7	Integrating open_clip, creating the FAISS index, and testing retrieval accuracy against historical plots.
GenAI Integration	8-9	Implementing the 4-bit VLM (LLaVA), integrating XGrammar constrained decoding, and testing JSON output reliability.
Frontend	10	React WebGL (uPlot) interactive timeline, ROI selection bounding-box, and dynamic JSON diagnostic UI cards.
Testing	11	XGrammar schema adherence checks, hallucination evaluation, and strict VRAM/RAM stress testing.
Deployment	12	Final local deployment, documentation finalization, and live demo preparation.

18. Team Responsibilities

Solo Developer (Full-Stack ML Engineer)

- **Product:** BRD/PRD/UX drafting, system architecture design, and enterprise pivot strategy.
- **Data Pipeline:** FastF1 extraction, Polars/DuckDB out-of-core streaming, and Diagnostic ROI visual cropping.
- **AI/ML:** CLIP embeddings, FAISS indexing, LLaVA 4-bit orchestration, XGrammar pushdown automata compilation, and GPU VRAM management.
- **Full Stack:** React WebGL (uPlot) observability dashboard, FastAPI backend development, API routing, and local deployment optimization.

19. Github Repository Structure

None

```
overcut-ai-f1-observability/
├── docs/
│   ├── 01-project-overview.md
│   ├── 02-brd.md
│   ├── 03-prd.md
│   ├── 04-hld-llld.md
│   └── 05-testing-deployment.md
├── frontend/                                # React, Tailwind CSS, uPlot (WebGL
Timeline)
├── backend/
│   ├── api/                                # FastAPI endpoints (e.g.,
/analyze_anomaly)
```

```
|   ├── core/                # Config, VRAM/RAM hardware checks,
logging                       # FastF1, Polars streaming, ROI
|   ├── data_engine/         # LLaVA orchestration, CLIP, XGrammar
cropping logic               # FAISS indexing and similarity
|   ├── ai/                  # Pydantic schemas (JSON output
compilation                  definitions)
|   ├── vector_store/        # FastF1 downloaded session data
search logic                 # Output micro-cropped PNGs for
|   └── models/              # Jupyter notebooks for testing
definitions)                # Pytest suite (VRAM checks, XGrammar
|   ├── data/               validation)
|   ├── raw_cache/          # Includes polars, duckdb, xgrammar,
|   └── generated_roi/      bitsandbytes
embeddings
|   ├── notebooks/
models and data streams
|   ├── tests/
validation)
|   ├── .env.example
|   ├── requirements.txt
|   ├── README.md
|   └── .gitignore
```